Estruturas de Dados I
Professor: Daniel Gomes Soares
Curso: Ciência da Computação – 2026
Trabalho 2: Análise de Métodos de Ordenação
Entrega: 06/07/2026
Trabalho individual

1. Objetivo

O objetivo deste trabalho é implementar, analisar e comparar diferentes algoritmos de ordenação, considerando seu funcionamento, desempenho prático e complexidade computacional.

O estudante deverá avaliar algoritmos clássicos apresentados em aula e também pesquisar dois algoritmos adicionais de ordenação, não abordados diretamente durante a disciplina. A análise deverá considerar diferentes tamanhos de entrada e diferentes configurações dos dados, permitindo observar o comportamento dos algoritmos em situações distintas.

2. Algoritmos que devem ser analisados

Devem ser implementados e comparados os seguintes algoritmos de ordenação:

- Bubble Sort;
- Insertion Sort;
- Selection Sort;
- Merge Sort;
- Quick Sort;
- Dois algoritmos de ordenação que não foram apresentados nas aulas.

Os dois algoritmos adicionais devem ser escolhidos pelo estudante e devidamente explicados no relatório. Exemplos possíveis incluem Heap Sort, Shell Sort, Counting Sort, Radix Sort, Bucket Sort, Timsort ou outro algoritmo de ordenação relevante.

3. Implementação

A implementação deverá ser feita na linguagem Java.

Cada algoritmo deverá ser implementado de forma separada e organizada, permitindo sua execução individual. O código deverá conter comentários indicando claramente:

- Onde ocorrem as comparações entre elementos;
- Onde ocorrem as trocas ou movimentações de elementos;
- Como é feita a divisão do problema, quando o algoritmo utilizar recursão;
- Como ocorre a combinação dos resultados, quando aplicável.

Para que a comparação seja justa, todos os algoritmos devem ordenar os mesmos conjuntos de dados. Assim, recomenda-se gerar um vetor original e criar cópias desse vetor para cada algoritmo testado.

4. Cenários de teste

Os algoritmos deverão ser testados com vetores contendo:

- 100.000 elementos;
- 1.000.000 de elementos.

Para cada tamanho de vetor, deverão ser considerados três cenários:

- Vetor já ordenado;
- Vetor em ordem inversa;
- Vetor com valores aleatórios.

Para cada execução, deverão ser coletadas as seguintes informações:

- Quantidade de comparações;
- Quantidade de trocas ou movimentações;
- Tempo de execução.

O tempo de execução deverá considerar apenas o processo de ordenação, não incluindo o tempo de criação ou preenchimento do vetor.

5. Relatório técnico

O relatório deverá conter, no mínimo, as seguintes seções:

a) Capa

A capa deve apresentar:

- Nome da instituição;
- Curso;
- Disciplina;
- Nome do professor;
- Título do trabalho;
- Nome do estudante;
- Data de entrega.

b) Introdução

A introdução deve apresentar o objetivo do trabalho, a importância dos algoritmos de ordenação na computação e uma breve contextualização sobre a análise de desempenho.

c) Descrição dos algoritmos

Nesta seção, devem ser explicados os algoritmos analisados.

Para os algoritmos apresentados em aula, a explicação pode ser mais objetiva. Para os dois algoritmos novos, a explicação deve ser mais detalhada, incluindo:

- Ideia principal do algoritmo;
- Etapas de funcionamento;
- Exemplo simples de execução;
- Vantagens e limitações;
- Situações em que o algoritmo pode ser adequado.

d) Metodologia dos testes

O relatório deve explicar como os testes foram realizados, incluindo:

- Configuração básica do computador utilizado;
- Critério de geração dos vetores;
- Tamanhos dos vetores;
- Tipos de entrada analisados;

- Forma de medição do tempo;
- Quantidade de execuções realizadas para cada teste.

Recomenda-se executar cada teste mais de uma vez e apresentar a média dos tempos obtidos, reduzindo a influência de variações ocasionais do sistema.

e) Gráficos e tabelas de desempenho

Devem ser apresentados gráficos e/ou tabelas comparando os algoritmos em relação a:

- Quantidade de comparações;
- Quantidade de trocas ou movimentações;
- Tempo de execução.

Os resultados devem ser organizados separadamente para cada tipo de entrada:

- Vetor ordenado;
- Vetor invertido;
- Vetor aleatório.

Os gráficos devem possuir título, legenda quando necessário, identificação dos eixos e unidades de medida.

f) Análise dos resultados

O estudante deverá comentar os resultados obtidos, relacionando-os com o funcionamento e a complexidade dos algoritmos.

A análise deve responder, entre outras, às seguintes questões:

- Quais algoritmos tiveram melhor desempenho em cada cenário?
- Algum algoritmo apresentou comportamento muito diferente dependendo da entrada?
- Os resultados práticos confirmam a complexidade teórica esperada?
- Quais algoritmos foram mais afetados pelo vetor invertido?
- Quais algoritmos se mostraram mais adequados para grandes volumes de dados?
- Os dois algoritmos pesquisados apresentaram desempenho competitivo em relação aos algoritmos estudados em aula?

g) Código-fonte

O código-fonte deve ser entregue junto com o relatório.

O código deve estar organizado, indentado e comentado. Também deve permitir a reprodução dos testes apresentados no relatório.

h) Conclusão

A conclusão deve apresentar uma síntese dos principais resultados obtidos.

O estudante deve indicar quais algoritmos considera mais adequados para diferentes situações, considerando tamanho da entrada, tipo de vetor, tempo de execução, número de operações e complexidade computacional.

6. Entrega

A entrega deverá conter:

- Relatório técnico em PDF;
- Link do GitHub com o código-fonte completo;